QUENYX VOPS HUB

INTERNAL

Developer Guide

Quenyx vOPS HUB — v1.0.0 · Document v2.1

Document Version	2.1
Software Version	v1.0.0
Applies To	Quenyx vOPS HUB v1.0.0
Classification	Internal
Owner	Platform Engineering
Status	Released
Last Updated	2026-06-30
Document Type	Developer guide

REVISION HISTORY

Version	Date	Notes
1.0	2026	Initial v1 pack (through Sprint 19).
2.0	2026-06-29	Aligned to v1.0.0; QynCore internal communication; Unified AI Workspace patterns.
2.1	2026-06-30	Added the Operations Intelligence (Sprint 21) pattern: consuming the shared AI runtime from a module without duplicating AI logic.

Table of Contents

11 — Developer Guide

Audience: New engineers. **Goal:** Get productive in the Quenyx monorepo and ship changes that match platform conventions.

---

1. Repository structure

```
quenyx-saas/
├─ backend/      Laravel API (PHP 8.3) — services, QCIF engines, AI platform
│  ├─ app/Http/Controllers/**      thin controllers
│  ├─ app/Services/**              business logic (Compliance, Ai, QuenyxAI)
│  ├─ app/Contracts/**             interfaces (e.g. AiModuleAdapterInterface)
│  ├─ app/Models/**                Eloquent models
│  ├─ config/**                    ai.php, compliance.php, quenyx_ai.php, …
│  ├─ database/migrations/**       65 migrations
│  ├─ database/seeders/**          incl. ComplianceCorpusSeeder
│  └─ routes/**                    api.php + per-domain route files
├─ frontend/     React + Vite + TS — SPA, platformRegistry.ts
├─ gateway/      Node edge/proxy
├─ agent/        QynSight host agent
└─ docs/         documentation (this pack: docs/quenyx-v1/)
```

2. Coding standards

- **Controllers are thin**: validate → authorize → delegate to a service → return a resource/array.
- **Business logic lives in services** under `app/Services/**`; keep them deterministic and testable.
- **UUIDs/codes** in public payloads for domain entities — never leak raw auto-increment IDs.
- **No hardcoded model names**; resolve from config/env.
- **Fail closed**: invalid input or missing entitlement → reject, don't degrade silently.
- Follow PSR-12 for PHP; the React side follows the repo's ESLint/TS config.

3. Service patterns

- One service = one responsibility (e.g. `ComplianceRetrievalService`, `RagIndexService`).
- Services receive dependencies via constructor injection (bound in `AppServiceProvider`).
- Cross-cutting AI access goes through `AiProviderRegistry` only.

4. Controller / resource patterns

- Group routes by domain in `routes/<domain>.php`, required from `routes/api.php`.
- Apply middleware at the group level: `auth:sanctum` + `project.qynshield` / `project.module:*` + the right named throttle.

- Return JSON arrays or Eloquent API Resources; keep response shapes stable (docs derive from them).

5. Migrations

- New migration per change; **never edit an applied migration** — fix forward.
- UUID primary keys for new domain entities; add indexes for query paths; preserve immutability for corpus tables.

6. Seeders

- Core data via `DatabaseSeeder`; corpus via `ComplianceCorpusSeeder` + `compliance:seed-source-documents`.
- **No fake data** — corpus content must come from official source documents and pass the validator.

7. Tests

- `php artisan test` (feature + unit). Filters: `--filter=Compliance`, `--filter=Ai`.
- Write tests for new services (deterministic inputs → deterministic outputs) and for route authorization/entitlement.

8. How to add a module

1. Add the module to the **frontend** `platformRegistry.ts` (keep it registered; sidebar visibility is controlled by the flag — don't re-enable hidden modules without product sign-off).
2. Add backend entitlement handling (module key, subscription) and `project.module:<key>` middleware if it needs gating.
3. Register the module in the backend AI catalog (`config/quenyx_ai.php`) with its AI readiness.

9. How to add an AI adapter

1. Implement `App\Contracts\QuenyxAI\AiModuleAdapterInterface` (`moduleKey`, `supportedSkills`, `supportedContexts`, `buildContext`, `buildReasoning`, `buildPrompt`).
2. Reuse existing module services — **do not duplicate business logic**.
3. Register it at boot: `QuenyxAiPlatform::registerAdapter($adapter)` in `AppServiceProvider::boot`.
4. Its `module_catalog` status flips to `production`; the capability catalog updates automatically.

10. How to add a skill

1. Implement the skill class under `app/Services/Ai/Skills/**`; it must **reuse a deterministic service** and make **no AI call**.

2. Register it in `config/ai.php` → `skills.registered` with a key, priority, and `enabled` flag.

11. How to add a compliance framework

1. Prepare official **source documents** + a manifest under `database/corpus/<authority>/<framework>/`.
2. Seed via `ComplianceCorpusSeeder` and `compliance:seed-source-documents --framework=… --release=…`.
3. The validator enforces no fake data, code uniqueness, and provenance. Create an active **revision**.

12. How to add API endpoints

1. Add the route to the relevant `routes/<domain>.php` under the correct middleware group.
2. Implement the controller method (thin) + the service.
3. Run `php artisan route:list` to confirm registration and **no collisions**.
4. Update Doc 08 (API Reference).

13. How to write documentation

- Update the affected doc(s) in `docs/quenyx-v1/` in the **same PR** as the code.
- Respect the status legend (/ / /). **No fabrication.**
- Re-derive Doc 08 (routes) and Doc 09 (migrations) when those change.

14. Definition of Done

- [ ] Code matches service/controller conventions; no hardcoded models; UUIDs in payloads.
- [ ] Routes registered, gated, throttled; `route:list` clean.
- [ ] Migrations are new (not edited); apply cleanly.
- [ ] Tests added/updated and passing (`php artisan test`).
- [ ] AI stays off-by-default; no direct provider calls outside provider classes.
- [ ] Docs updated; status badges correct.
- [ ] No fake/sample data; corpus validator passes.

Working on Quenyx AI (Unified AI Workspace — Sprint 20)

> **v1.0.0:** the UI label is **Quenyx AI**. The canonical SPA base stays `/ai-workspace/*` (a `/quenyx-ai/*` alias in `App.tsx` redirects to it). The provider catalog is declared in `App\Services\Ai\AiProviderCatalog`; `AiProviderRegistry` decides which entries are executable (have an adapter) and which are platform-configured. Keep new catalog entries non-executable until a real adapter is added — never fabricate connectivity.

- **Backend** lives in `App\Http\Controllers\Ai\Workspace`, `App\Services\Ai\Workspace`, `App\Models\Ai\*`, `app/Http/Resources/Ai/*`, and `routes/ai-workspace.php`. Resolve + authorize a workspace through `AiWorkspaceBaseController::workspace()` (which calls `AiWorkspaceContextResolver` and `ProjectPolicy::accessAi|administerAi`), then gate fine-grained actions with `requireCapability( … )`. Return data via the `{ success, data }` envelope (`ok()`); expose **UUIDs only**.
- **Reuse, don't duplicate**: conversations via `AiConversationRepository`; execution via `AiProviderRegistry` + `CompliancePromptOrchestrator`; skills/capabilities via `QuenyxAiPlatform`. New AI logic does not belong here.
- **Secrets**: store on `AiProviderSetting.settings` (the `encrypted:array` cast) and never return raw values — only `secret_configured`. Audit via `AiWorkspaceAuditLogger` (it strips secrets).
- **Frontend**: add a tab in `layouts/AiWorkspaceLayout.tsx`, a lazy route in `App.tsx`, a page under `pages/ai/*` using `useAiResource` + `AiView` (handles no-workspace/loading/error/empty), a method on `services/aiWorkspaceService.ts`, types in `types/aiWorkspace.ts`, and matching keys in **both** the `en` and `ar` blocks of `i18n/translations.ts`.
- **Validate**: `php artisan route:list | grep ai`, `php artisan migrate --force`, `php artisan test`, `npm run build` (+ `npm run lint`).

Consuming the AI runtime from a module — Operations Intelligence (Sprint 21)

Sprint 21 (QynSight Operations Intelligence) is the reference pattern for making a module a **live AI consumer without duplicating AI logic**. Follow it when adding intelligence to any module.

- **One integration point.** Route *all* model access through a single module service that wraps the shared runtime — here `App\Services\Observe\Intelligence\OperationsAiAnalyst`, which uses `AiProviderRegistry` (providers), `CompliancePromptOrchestrator` (grounded prompt + citations), `AiConversation(Message)` / `AiConversationRepository` (conversations), and `AiAccessAuditLogger` (audit). **Do not** add a new provider registry, prompt/reasoning/RAG engine.
- **Deterministic evidence first.** Build deterministic, real-data evidence in plain services (`OperationsEvidenceCollector`, `RootCauseService`, `IncidentTimelineService`, `CapacityAdvisorService`, `InfrastructureImpactService`, `PerformanceAdvisorService`, `AlertExplanationService`, `OperationsIntelligenceService`) and pass it to the analyst to *render*. Never let the model invent facts; surface "insufficient evidence" explicitly.
- **UUID-only over numeric ids.** When a module stores numeric ids, derive a deterministic **UUIDv5** (`App\Support\Observe\OperationsEntityId`, namespaced by type + workspace + id) and resolve it back with a workspace-scoped resolver (`OperationsEntityResolver`). No schema change; never expose numeric ids.
- **Security per request.** Use a base controller (`OperationsIntelligenceBaseController`) that resolves the workspace by UUID and enforces module entitlement (`qynsight`) + monitoring RBAC (`ProjectPolicy::accessAi`) + the `can_use_ai` capability, then audits. Routes go in their own file (`routes/qynsight-intelligence.php`) under `auth:sanctum` + `throttle:ai-workspace`.

- **Frontend.** Reuse `apiClient` + the `useAiWorkspaceUuid` / `useAiResource` hooks, a typed service (`operationsIntelligenceService`), and a contextual `QuenyxAiButton` that opens a copilot drawer backed by a real Quenyx AI conversation. Add **both** `en` and `ar` keys in `i18n/translations.ts`.
- **Validate**: `php -l` the new PHP files, `php artisan route:list | grep qynsight/intelligence`, `php artisan test --filter=OperationsIntelligence`, and `npm run build` / `npm run lint`.

Adding AI to a module (the AI Adapter Platform — Sprint 22)

Module AI is now a **plug-in**. To make any module an AI consumer:

1. **Implement** an `App\Contracts\QuenyxAI\AiModuleAdapter` by extending `AbstractAiModuleAdapter` (declare `moduleKey`, metadata, `capabilities()`, `availableActions()` with UUID-only endpoints, and `buildContext()` from your **real** domain data).
2. **Build evidence + narrate.** Put deterministic evidence in module services and narrate via the shared `App\Services\Ai\ModuleAiNarrator` — **never call a provider directly** and never duplicate provider/prompt/reasoning/RAG logic.
3. **Register** the adapter once in `AppServiceProvider::boot()`: `$registry->register($this->app->make(MyModuleAiAdapter::class));`
4. **Expose** UUID-only, workspace-scoped routes behind the standard envelope (copy `AssetIntelligenceBaseController`), and reuse the generic `AiCopilotDrawer` on the frontend.

That's it — the discovery APIs (`/api/ai/adapters`, `/api/ai/actions`), capability aggregation, and the AI Workspace surface your module automatically. Reference implementation: **QynAsset** (`app/Services/Asset/Intelligence/*`, `QynAssetAiAdapter`, `routes/qynasset-intelligence.php`). Full walkthrough in the **AI Adapter Developer Guide (Doc 23)**.

Adding an execution adapter (Automation Platform, Sprint 23)

Execution adapters plug into the **Automation Platform** the same way AI adapters plug into the AI registry — no core change. To add a runner (e.g. Docker, AWS):

1. Implement `App\Contracts\Automation\ExecutionAdapter` (or extend `AbstractExecutionAdapter` / `AbstractHttpExecutionAdapter`). Declare `key()`, `capabilities()`, `parameterSchema()`, `supportsRollback()`, `isOperational()`, and `execute()` / `rollback()`.
2. **Be honest & safe:** return `ExecutionResult::dryRun(...)` unless `liveAllowed()` is true, the target passes the allowlist, and the runner is enabled in `config/automation.php`. If a live runner isn't provisioned, return `ExecutionResult::skipped(...)` — never fabricate output.
3. Register it in `AppServiceProvider::boot()` via `AutomationAdapterRegistry::register(...)`. Add actions to `config('automation.actions')` (mark `destructive` / `rollback`).

The Execution Engine handles dry-run defaulting, approval gating, retries, audit, and learning for you. The Library UI, workflows, and runbooks pick up the new adapter automatically. See the **Automation Platform Guide (Doc 24)**.

Adding a knowledge source (Knowledge Platform, Sprint 24)

Knowledge sources plug into the `KnowledgeSourceRegistry` the same way — no core change to Enterprise Search, the Knowledge Graph, or the Global Timeline. To add a provider (e.g. Confluence, a Vector Store):

1. Implement `App\Contracts\Knowledge\KnowledgeSource` (or extend `AbstractKnowledgeSource`). Declare `key()`, `name()`, `isOperational()`, `search()`, and `count()`.
2. **Be honest:** until the provider is actually wired, return `isOperational() === false` (this is what `PlannedKnowledgeSource` does) — Enterprise Search will skip it cleanly rather than fabricate hits. Operational sources must return only **real indexed rows**.
3. Register it in `AppServiceProvider::boot()` via `KnowledgeSourceRegistry::register(…)`.

Enterprise Search, the registry view (`GET /api/qynknow/sources`), and the UI pick up the new source automatically — **no provider-specific branching anywhere**.

Adding collaboration to a new surface (Sprint 24)

Collaboration is a shared, polymorphic capability. To add comments/mentions/watchers/assignees/owners to any entity: call `CollaborationService` server-side (addressed by `entity_type` + `entity_uuid`) and drop the reusable `CollaborationPanel` React component into the page with `{ workspaceUuid, entityType, entityUuid }`. Do **not** build a per-module comment system. New AI surfaces must narrate only through `ModuleAiNarrator` and register an `AiModuleAdapter`. See Docs 28–32.

Publishing & subscribing to platform events (Event Bus, Sprint 25)

Modules must **not** call each other directly. To announce something happened, publish a domain event:

1. Use a canonical name from `App\Services\Platform\EventBus\PlatformEventNames` (typos throw). Then: `app(PlatformEventBus::class)->publish(PlatformEventNames::INCIDENT_OPENED, $project, $user, […payload], $correlationId)`. The publish is workspace-aware and audited (`platform_event_published`); fan-out is isolated.
2. To **react**, implement `App\Contracts\Platform\EventSubscriber` (`key()`, `subscribedTo()`, `handle()`; return `['*']` to receive everything) and register it once in `AppServiceProvider::boot()` via `PlatformEventBus::subscribe(…)`. The publisher is untouched — no module branching. See Doc 35.

Building AI context (Context Engine, Sprint 25)

Never hand-assemble enterprise context. Call `app(EnterpriseContextEngine::class)->build($project, $user, ['query' => …, 'exclude' => […]])` to get one normalized object (workspace, user, permissions, cross-module gather, timeline, graph, search). It is a read-model and recursion-safe. QynVA (`QynVaOperatorService`) consumes it to reason and propose **editable** cross-

module plans — and **never executes**. New module intelligence should follow the same pattern: deterministic evidence first, then narrate via `ModuleAiNarrator`. See Docs 34, 37.